

Backend Migration

Express Server -> Next.js API Routes

For Vercel Deployment

September 2026

1. The Problem

The project had a two-process architecture:

- A Next.js frontend running on port 3000
- A standalone Express server (backend/server.js) on port 4001

The frontend called `http://localhost:4001` for all API requests. Vercel only runs serverless functions and cannot host a persistent Express process. This meant the entire backend was non-functional in production.

2. The Solution

All backend logic was moved into Next.js API route handlers. The frontend and backend now run as a single Next.js application. Vercel automatically deploys the API routes as serverless functions.

3. New Files Created

lib/api/store.ts - Shared Product Store

A shared in-memory Map that stores product records. Both the products route and the status route import from this single module so they share state within the same serverless instance.

```
export interface StoredProduct {
  id: string;
  name: string;
  taskId: string;
  status: string;
  mode: 'single' | 'multiview';
  angles: string[];
  modelUrls: { glb: string } | null;
  thumbnailUrl: string | null;
  progress: number;
  createdAt: string;
}
export const products = new Map<string, StoredProduct>();
```

app/api/products/route.ts - POST + GET

POST /api/products - Accepts multipart form data with 1-4 photos, a product name, and angle metadata. Uploads each photo to the Tripo3D API, creates a 3D generation task, stores the product in the shared Map, and returns the productId and taskId.

GET /api/products - Returns all stored products as JSON. Used by the storefront page.

Key changes from the Express version:

- Replaced multer with native `Request.formData()` (Web API)
- Replaced `fs.writeFileSync` with returning Tripo CDN URLs directly
- Replaced Express `res.json()` with `NextResponse.json()`

- Uses Next.js module-level Map instead of a plain object

app/api/products/[id]/status/route.ts - GET

GET /api/products/:id/status?taskId=xxx - Polls the Tripo3D API for task status using the taskId query parameter. Returns the current status, progress percentage, and on success, the Tripo CDN URL for the .glb model file.

Key design decision:

The status route requires a taskId query parameter and queries Tripo directly. It does NOT depend on the in-memory Map for status lookups. This means status polling works reliably even across cold starts. The Map is updated opportunistically if the product happens to be in memory, but it is not required.

app/api/auth/login/route.ts - Placeholder

POST /api/auth/login - Accepts email and password, returns a placeholder response. This endpoint did not exist in the Express server either; it was called by the frontend but never implemented. Ready for real authentication logic to be added.

app/api/auth/signup/route.ts - Placeholder

POST /api/auth/signup - Accepts user details, returns a placeholder response. Same situation as login - called by frontend but never implemented in Express.

app/api/v1/nafdac/verify/route.ts - Placeholder

POST /api/v1/nafdac/verify - Accepts a NAFDAC registration number, returns a placeholder verification record. Called by the frontend's verification module but never existed in Express.

4. Files Modified

.env.local

Changed NEXT_PUBLIC_API_URL from http://localhost:4001 to an empty string. Since the API routes are now on the same domain as the frontend, no cross-origin URL is needed. The frontend's API client already handles empty string as same-origin.

```
# Before
NEXT_PUBLIC_API_URL=http://localhost:4001

# After
NEXT_PUBLIC_API_URL=
```

lib/api/auth.ts

Updated the base URL for login and signup fetch calls. Changed from using the raw NEXT_PUBLIC_API_URL (which was the Express server URL) to using the /api prefix that matches the new Next.js route structure.

```
# Before
fetch(`${API_URL}/auth/login`, ...)
fetch(`${API_URL}/auth/signup`, ...)

# After
fetch(`${API_URL}/api/auth/login`, ...)
```

```
fetch(`${API_URL}/api/auth/signup`, ...)
```

lib/api/products.ts - getProductGenerationStatus

Added an optional taskId parameter. The frontend now passes the taskId (received from the createProduct response) as a query parameter to the status endpoint. This decouples status polling from the in-memory product store.

Before

```
getProductGenerationStatus(productId: string)
```

After

```
getProductGenerationStatus(productId: string, taskId?: string)
```

features/seller/new-listing.tsx

Updated pollGeneration to accept and forward the taskId. The onSuccess callback now passes both result.productId and result.taskId.

Before

```
pollGeneration(result.productId)
```

After

```
pollGeneration(result.productId, result.taskId)
```

5. Architectural Changes

File uploads

Before: multer (memoryStorage) handled multipart parsing in Express.

After: Next.js Request.formData() is a native Web API. Files are accessed via formData.getAll('photos') and converted to ArrayBuffer for upload to Tripo.

3D model file storage

Before: The Express server downloaded .glb files from Tripo and saved them to backend/public/generated/ using fs.writeFileSync(). The frontend fetched them from /generated/p_123.glb.

After: The Tripo CDN URL is returned directly to the frontend. The resolveBackendUrl() function in lib/api/products.ts already handles absolute URLs (passes them through unchanged), so no frontend change was needed for model loading.

Status polling

Before: Frontend sent productId -> Express looked up taskId in memory -> queried Tripo -> returned result.

After: Frontend sends both productId and taskId -> API route queries Tripo directly with the taskId -> returns result. No Map lookup required. Works across cold starts.

Product store

Before: const products = {} (plain object, lost on restart).

After: Map<string, StoredProduct> in lib/api/store.ts (module-level, shared between route files). Still in-memory, so the storefront listing (GET /api/products) may be empty after cold starts. Requires a database for full persistence.

6. Deployment Checklist

- Add TRIPO_API_KEY to Vercel environment variables (Settings -> Environment Variables)
- The backend/ directory is no longer needed for deployment (can be kept for local dev reference)
- Remove NEXT_PUBLIC_API_URL from Vercel env vars if it was set there (empty string is the default)

7. Known Limitations

- Product listing (GET /api/products) is in-memory. Data is lost between cold starts. Fix: add Vercel KV, Postgres, or similar.
- Auth endpoints (login, signup) are placeholders. No real authentication is implemented yet.
- NAFDAC verification endpoint is a placeholder. No real verification logic is implemented yet.
- The in-memory Map means two concurrent users may not see each other's products on the storefront.

8. API Route Summary

Route	Method	Purpose
/api/products	POST	Create product + start 3D gen
/api/products	GET	List all products (storefront)
/api/products/[id]/status	GET	Poll Tripo task status
/api/auth/login	POST	User login (placeholder)
/api/auth/signup	POST	User signup (placeholder)
/api/v1/nafdac/verify	POST	NAFDAC verification (placeholder)

9. New File Tree (API routes)

```
app/api/  
  products/  
    route.ts          # POST + GET  
    [id]/  
      status/  
        route.ts      # GET (taskId query param)  
  auth/  
    login/  
      route.ts        # POST (placeholder)  
    signup/  
      route.ts        # POST (placeholder)  
  v1/  
    nafdac/  
      verify/  
        route.ts      # POST (placeholder)  
lib/api/  
  store.ts           # Shared product Map
```